Initial Architecture Document

StyleView Mobile Web App

Team Number	13
Team Members	Ben Haney Bradley Brown Caleb Hite Charlie Doherty Kai Barnhart Will Calhoun
Project Name	StyleView (work in progress)
Project Synopsis	A mobile web app that uses the phone's front camera to overlay shirts and hairstyles on the user in real time

Architecture Description

Overview

StyleView is a mobile web application that runs entirely in the phone's browser. It shows a live, mirrored view from the front camera and draws a virtual shirt or hairstyle over the user in real time. There is no application server. Video processing, image editing, and storage all happen on device, and a free static web host only delivers the app files and tracking models.

Architectural Style

The system is a layered, client-side architecture built around the frame-processing pipeline. The presentation layer is the UI Shell and Shirt Studio. The service layer holds the Camera, Tracking, Stabilizer, Rendered, Asset, and Storage services. The platform layer consists of free browser APIs and the open-source MediaPipe Tasks Vision library. Services communicate through small interfaces, so a component such as the tracker can be replaced without changing the renderer. Figure 1 displays the components and their dependencies.

Component Responsibilities

UI Shell: mode selection, camera-permission help, placement guidance, disclosures, and saved items.

Camera Service: wraps the browser camera API, handles denied permission, handles rotation, owns mirroring, and stops the camera when the user leaves the preview.

Tracking Service: loads the model for the active mode (Face Landmarker for hairstyles, Pose Landmarker for shirts), reports loading and failure, and supports retry.

Stabilizer: smooths landmarks with a filter, hides the overlay when tracking is lost, and restores it when tracking returns.

Renderer: warps the overlay onto anchor points, composites it over the camera frame on a canvas, and applies user size and position adjustments.

Asset Service: provides the sample shirt, three preset hairstyles, and user shirts, each with anchor points.

Shirt Studio: captures or uploads a shirt photo, validates it, and lets users crop, rotate, trace the outline, and mark key points

Storage Service: keeps shirts and preview pictures in database and supports deletion.

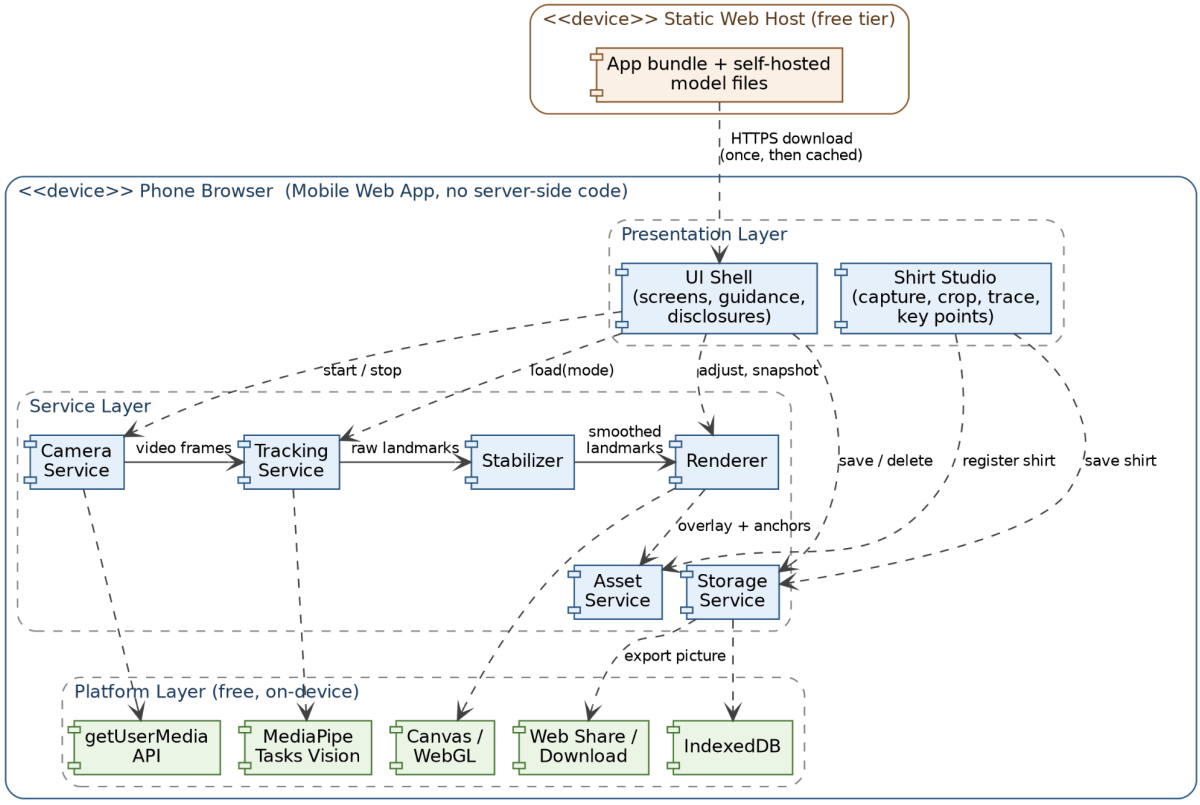


Figure 1. UML component diagram: structure and dependencies of mobile web app.

Runtime Behavior

Figure 2 shows the main runtime flow. The user selects a mode and starts the preview. The Camera Service requests permission, and if the user denies it, the UI Shell explains how to re-enable the camera. Then, the Tracking Service loads the matching model and offers a retry if loading fails. For every video frame, the Camera Service passes the image to the Tracking Service, which returns raw landmarks. The Stabilizer smooths them and reports whether the body or face is visible. The Renderer warps and composites the overlay, mirroring the output one, and draws it to the canvas. Because shirts and hairstyles are mutually exclusive modes, only one model runs at a time, which helps frame rate. Taking a picture sends a snapshot to the Storage Service, and leaving the preview stops the camera.

Privacy and Security

Camera frames are never serialized or sent over a network. A Content Security Policy restricts network connections to the app’s own origin, making accidental uploads impossible. Model files are self-hosted and saved data stays within the browser’s DB. Users have the ability to delete individual items or clear everything.

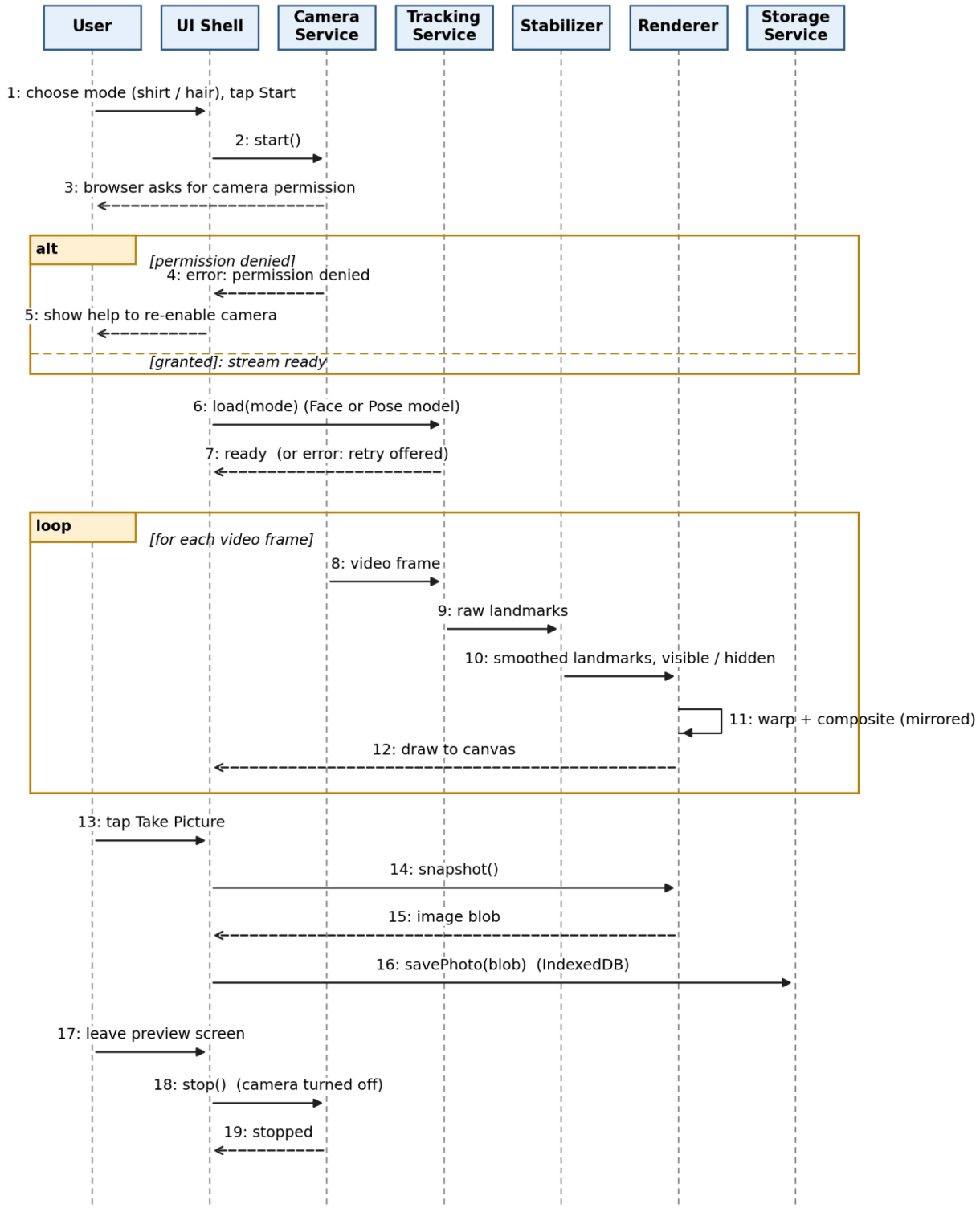


Figure 2: UML sequence diagram: camera start, per-frame try on loop, snapshot, and shutdown

Data and Class Structure

Figure 3 shows the main classes. TryOnSession coordinates the CameraService, Tracker, Stabilizer, and Renderer. TryOnSession also holds the current Mode, TrackingState, and user Adjustment. FaceTracker and PoseTracker both implement the Tracker interface. Garment and HairStyle both implement the Overlay interface and each carry an AnchorSet, so the sample shirt, user-made shirts, and hairstyles all follow one rendering path. StorageService manages Garment and SavedPhoto records.

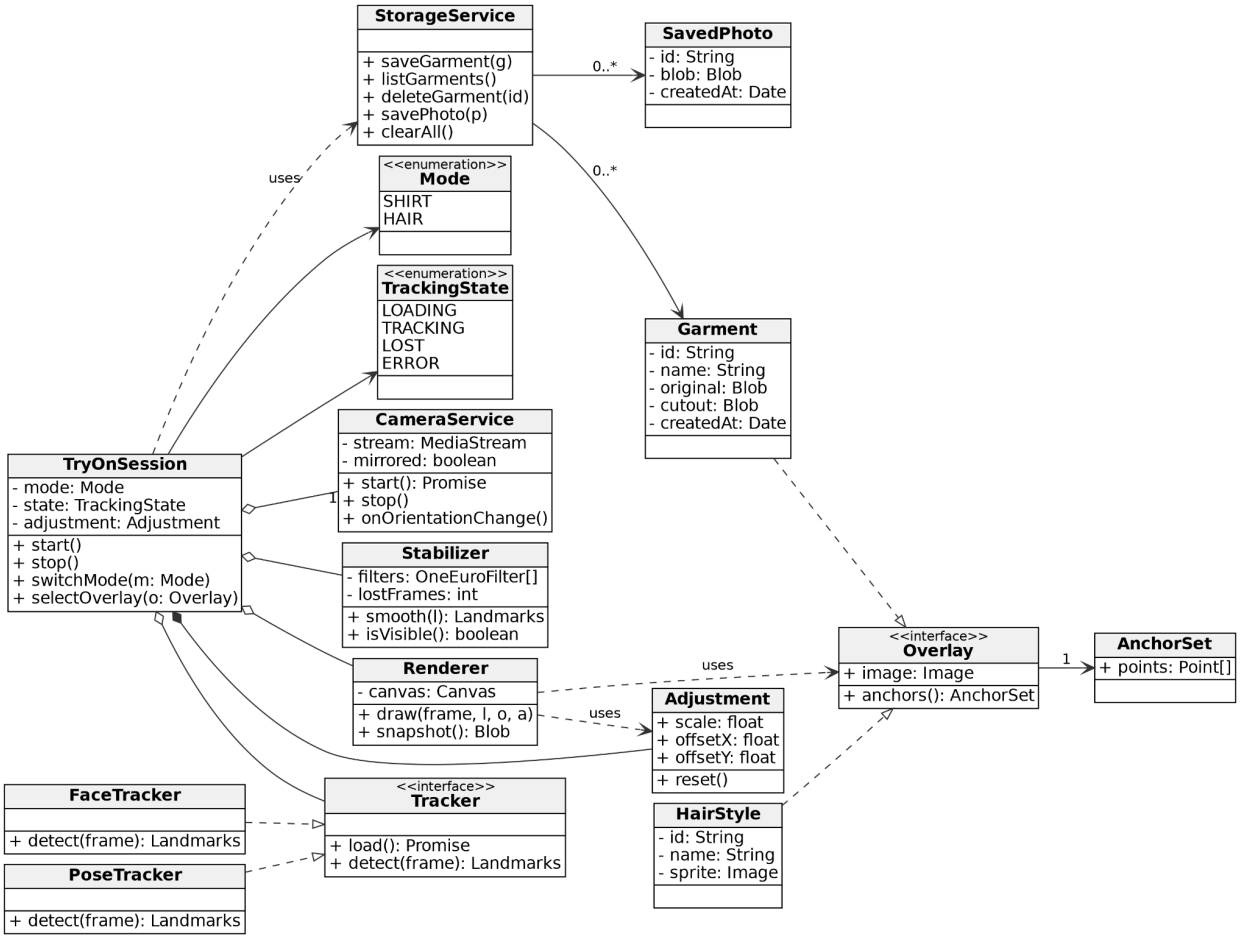


Figure 3: UML class diagram: session, tracking, rendering, overlay, and storage classes.

Key Decisions and Risks

A mobile web app was chosen over native apps because one codebase covers both iPhone and Android devices. Two-dimensional sprite warping replaces three-dimensional meshes because the app shows appearance, not fit. The main risks are frame rate on the test phones, browser camera behavior, and keeping hands and arms in front of the shirt, which is treated as a stretch goal.